

CSC 313: Formal Methods – Final Project (10 marks)

Case Study: Cafeteria Ordering System

In your CSC 310 course you built a JavaFX Cafeteria Ordering System supporting customer and manager roles, menu browsing, cart management, order placement, and manager inventory control. For this project you will write a formal specification in VDM-SL that captures the essential logical state and operations of that same system. Ignore the GUI, password hashing, database, and networking – focus solely on the data and the rules that govern valid system behaviour.

Project Requirements

Use the VDM-SL constructs covered in class to model the following types, state, invariant, initial state, and operations. All specifications must be handwritten (see submission rules).

Types (use token, records, maps, etc.)

- `CustomerId`, `ManagerId`, `MenuItemId` – distinct token types (automatically disjoint).
- **Category** – enumerated type: `<Drinks> | <MainMeal> | <Swallow> | <Protein> | <Snacks>`.
- **MenuItem** – record: `id : MenuItemId, name : seq of char, category : Category, price : nat, quantityInStock : nat, description : seq of char`.
- **Cart** – map `MenuItemId` to `nat` (item id $\rightarrow$ quantity).
- **Order** – record: `customer : CustomerId, items : map MenuItemId to nat, total : nat`.

State

Define a state called `Cafeteria` with the following fields:

```
registeredCustomers : set of CustomerId
registeredManagers : set of ManagerId
menu : map MenuItemId to MenuItem
carts : map CustomerId to Cart
orders : set of Order
```

Invariant (at least the following, expressed with inv)

- $\text{dom carts} \subseteq \text{registeredCustomers}$
- For every customer $\text{cid} \in \text{dom carts}$, $\text{dom carts}(\text{cid}) \subseteq \text{dom menu}$
- For every order $\text{o} \in \text{orders}$, $\text{o.customer} \in \text{registeredCustomers}$ and $\text{dom o.items} \subseteq \text{dom menu}$
- Note: CustomerId and ManagerId are different token types, so the sets $\text{registeredCustomers}$ and $\text{registeredManagers}$ are inherently disjoint – no extra clause needed.

Initial State (init)

All sets and maps are empty; no orders exist.

Operations to Specify (provide pre and post for each; you may use ext rd / ext wr)

1. RegisterCustomer

```
RegisterCustomer(cid : CustomerId)
```

Adds a new customer (provided the ID is unused) and initialises an empty cart for them.

2. ViewMenu

```
ViewMenu() result : map MenuItemId to MenuItem
```

Returns the current menu without changing anything.

3. AddToCart

```
AddToCart(cid : CustomerId, mid : MenuItemId, qty : nat)
```

Adds a chosen quantity of a menu item to the customer's cart. The quantity must be positive, the item must exist on the menu, and the total quantity (what is already in the cart plus the new amount) must not exceed the available stock.

4. PlaceOrder

```
PlaceOrder(cid : CustomerId)
```

Converts the customer's non-empty cart into an order. The cart quantities must not exceed the current stock. The operation computes the total price, adds an Order to the set of orders, decreases the stock of each ordered item, and clears the customer's cart.

5. AddMenuItem (Manager operation)

```
AddMenuItem(mgrId : ManagerId, mid : MenuItemId, name : seq of char, cat  
: Category, price : nat, stock : nat, desc : seq of char)
```

A registered manager adds a new menu item (the item ID must not already exist, price > 0, stock $\geq$ 0).

Submission Instructions

- Handwrite your entire specification on paper.
- Take a clear photograph or scan and assemble a single PDF file.
- Upload the PDF to the student portal by **Wednesday, June 3rd 11:59 PM** (late submissions will not be accepted).
- Write your full name and matriculation number on every page.
- This is individual work; any form of copying will be penalised according to university regulations.